Assignment 9.2 - Governance, Audit, and Controlled Orchestration

Part I — Governance & Audit Report

During Week 7, I used AI to implement an enhanced order-summary feature in the Virtual Taco Stand application. The requested feature replaced the original quantity-first presentation with a more structured line-item display, added generated identifiers such as “Item 1,” displayed additional pricing information, and added a Remove Taco button using Angular @Output and EventEmitter. The AI-generated implementation compiled successfully and appeared functional in the browser, but seven previously passing unit tests failed. Several failures involved exact wording and formatting assumptions, including quantity-first labels, the phrase “Price per taco,” the absence of Remove Taco buttons, and the absence of generated item identifiers. These failures demonstrated that visual success did not necessarily mean the system still satisfied its existing behavioral contracts.

The deeper review also exposed a true regression that was not obvious from the initial appearance of the feature. The child component emitted the taco’s menu ID when the user clicked Remove Taco, and the parent filtered the order using that ID. Because multiple order entries could share the same menu-item ID, removing one Carnitas entry could remove every Carnitas entry in the order. This revealed an important data-model assumption: a menu-item identifier had been treated as though it uniquely identified an order line. The Week 7 readings emphasized that component communication forms a contract between parent and child components and that a feature can compile and render while still passing incorrect information through that contract.

In Week 8, I separated the failures into true regression and outdated legacy expectations. The removal problem was a genuine application defect and was repaired by introducing a unique line-item identifier so each order entry could be targeted independently. By contrast, tests that required the old quantity-first format, exact price wording, no inline removal controls, and no generated item labels conflicted directly with the newly approved feature requirements. Those tests were updated intentionally rather than merely weakened to force a passing result. The revised tests focused more on meaningful behavior such as

displayed values, unique line-item removal, and correct event communication. The full test suite was rerun after the repairs, followed by manual testing of duplicate taco entries, totals, removal behavior, and the empty-order state. The Week 8 reading reinforces that tests should only be updated when expected behavior has legitimately changed and that validation must include both automated tests and application behavior.

This experience exposed how tightly some of the original tests were coupled to specific UI wording and DOM structure. Those tests were useful as witnesses during Week 7, but they also demonstrated how brittle tests can increase maintenance when legitimate requirements change. More resilient tests should validate business behavior, data flow, event payloads, and calculated results rather than unnecessarily freezing presentation details.

The larger lesson is that AI does not reduce developer responsibility. The developer remains responsible for architecture, correctness, regression prevention, testing, review, and documentation. AI-generated code should never be accepted simply because it compiles or looks reasonable. The Week 9 reading describes orchestration as defining a narrow task, explicitly limiting what AI may change, and verifying the result before integration. These safeguards allow AI to accelerate implementation while keeping engineering judgment and system ownership with the developer.